Gymnázium, Praha 6, Arabská 14
tel.: 235 351 708
fax.: 222 262 066
e-mail: ga@gyarab.cz
www.gyarab.cz

Zadání maturitní práce

Téma maturitní práce:	**Systém pro skupinovou komunikaci přes SMS**
Zadavatel:	**RNDr. Zdeňka Hamhalterová**
Řešitel:	**Vejražka Jakub, 4.E**
Vedoucí práce:	**Ing. Daniel Kahoun**
Oponent:	**Ing. Zdeněk Muzikář, CSc.**
Termín odevzdání:	**31. 3. 2026**

Způsob zpracování a kritéria hodnocení:

Zpracování se řídí obecně závaznými pokyny pro zpracování maturitních prací v požadovaném rozsahu. Řešitel odevzdá na studijním oddělení určenému zástupci vedení školy ve stanoveném termínu dva svázané výtisky projektové dokumentace s podepsaným prohlášením o autorství a jeden poster. Přílohu s dokumentací ve stejném znění, posterem, zdrojovými kódy a dalšími podklady řešitel odevzdá podle pokynů vedoucího práce také elektronicky. Hodnotí se odborné zpracování, užití návrhových vzorů, prezentace při obhajobě a funkcionalita produktu. Posudek vedoucího a oponenta hodnotící samotnou práci získá řešitel k nahlédnutí před obhajobou.

Popis (povinná část):

Realizujte komunikační platformu, která umožní skupinový chat v rámci standardních SMS zpráv. Služba je určena pro uživatele bez chytrých telefonů a zahrnuje webové rozhraní pro správu uživatelských účtů a administraci skupin.

Upřesnění zadání:

- Napojení webové aplikace na SMS bránu přes API
- Logika pro distribuci zpráv mezi členy
- Registrace a ověřování uživatelů pomocí telefonního čísla
- Portál pro správu kontaktů a skupin
- Ošetření limitů a front zpráv pro zajištění doručitelnosti

Bonus (nepovinná část):

- Implementace příkazů ovládaných přímo přes SMS (např. */join*)
- Podpora pro plánované odesílání hromadných oznámení
- Statistiky odeslaných zpráv a správa kreditního systému
- Šifrování citlivých dat v databázi aplikace

Platforma:

- CSS
- HTML
- JavaScript
- Linux
- Node.js

datum podpisu	podpis řešitele

Anotace

CZ

Ding je platforma, která zprostředkovává skupinové konverzace v SMS zprávách. To umožňuje využívání výhod skupinových chatů i v prostředích bez internetu, např. na tlačítkových telefonech. Systém stojí na webovém rozhraní, přes které lze spravovat skupiny a finance.

Primární cílovou skupinou projektu jsou uživatelé tl ačítkových telefonů, pro které v současné době neexistuje žádná podobná alternativa. Zároveň ale může být dobrou volbou pro lidi, kteří se dlouhodobě nachází mimo dosah internetu.

Službu si lze vyzkoušet zde: https://ding.kubavejrazka.dev

EN

Ding is a platform that facilitates group conversations via SMS messages. This makes it possible to enjoy the benefits of group chats even in environments without internet access, for example on feature phones (push-button phones). The system is built on a web interface through which groups and finances can be managed.

The project's primary target audience is feature phone users, for whom no similar alternative currently exists. It can also be a great option for people who are out of internet range for extended periods.

You can try out the service here: https://ding.kubavejrazka.dev

DE

Ding ist eine Plattform, die Gruppenunterhaltungen über SMS-Nachrichten vermittelt. Dies ermöglicht es, die Vorteile von Gruppenchats auch in Umgebungen ohne Internet zu nutzen, z. B. auf Tastenhandys. Das System basiert auf einer Weboberfläche, über die Gruppen und Finanzen verwaltet werden können.

Die primäre Zielgruppe des Projekts sind Nutzer von Tastenhandys, für die es derzeit keine vergleichbare Alternative gibt. Gleichzeitig kann es aber auch eine gute Wahl für Menschen sein, die sich über einen längeren Zeitraum außerhalb der Internetreichweite befinden.

Den Dienst können Sie hier ausprobieren: https://ding.kubavejrazka.dev

Obsah

1. Úvod ... 1
2. Použité technologie ... 1
3. UI a UX ... 1
   3.1. Z pohledu uživatele ... 1
   3.2. Implementace ... 4
4. Backend ... 4
   4.1. Hosting ... 4
   4.2. Databáze ... 5
      4.2.1. Uživatelé ... 5
      4.2.2. Skupiny ... 5
      4.2.3. Pozvánky do skupin ... 6
   4.3. Komunikace s SMS bránou ... 6
      4.3.1. Odesílání zpráv ... 6
      4.3.2. Příjem zpráv ... 7
   4.4. Logika aplikace ... 7
5. Prostor pro zlepšení ... 8
Zdroje ... 9

1. Úvod

Nápad na vytvoření tohoto projektu jsem dostal loni na hodině WA, kde nás učil Kamil Foltin, který nepoužívá chytrý telefon, a debatovali jsme o tom, jaké mají "hloupé" telefony limitace. Za jeden z poměrně výrazných nedostatků těchtotelefonů považuji absenci možnosti skupinové komunikace.

Vytvořil jsem tedy službu, která tento nedostatek řeší serverem, který distribuuje zprávy mezi členy skupiny a doplňuje k nim přezdívky odesílatelů, takže každý člen má ve svých SMS zprávách konverzaci podobnou skupinovým chatům např. na WhatsApp nebo Instagramu.

2. Použité technologie

Celý projekt stojí na frameworku *SvelteKit* [1]. Je v něm napsaný front-end i back-end webové aplikace. Veškerá back-endová logika je řešená v TypeScriptu.

Dále je pro provoz naprosto klíčová SMS brána. V aktuální verzi používám *BulkGate* [2], ale mám v plánu migrovat jinam (viz Prostor pro zlepšení, str. 8). Během vývoje jsem také použil platformu *SmsManager* [3].

Web hostuji na virtuálním stroji u *Oracle Cloud* [4], během vývoje jsem původně využíval VM u *Google Cloud* [5], ale přesunul jsem se, jelikož Oracle nabízí lepší výkon za nižší cenu. Přes *Google Cloud* nyní řeším pouze automatizované odesílání emailů.

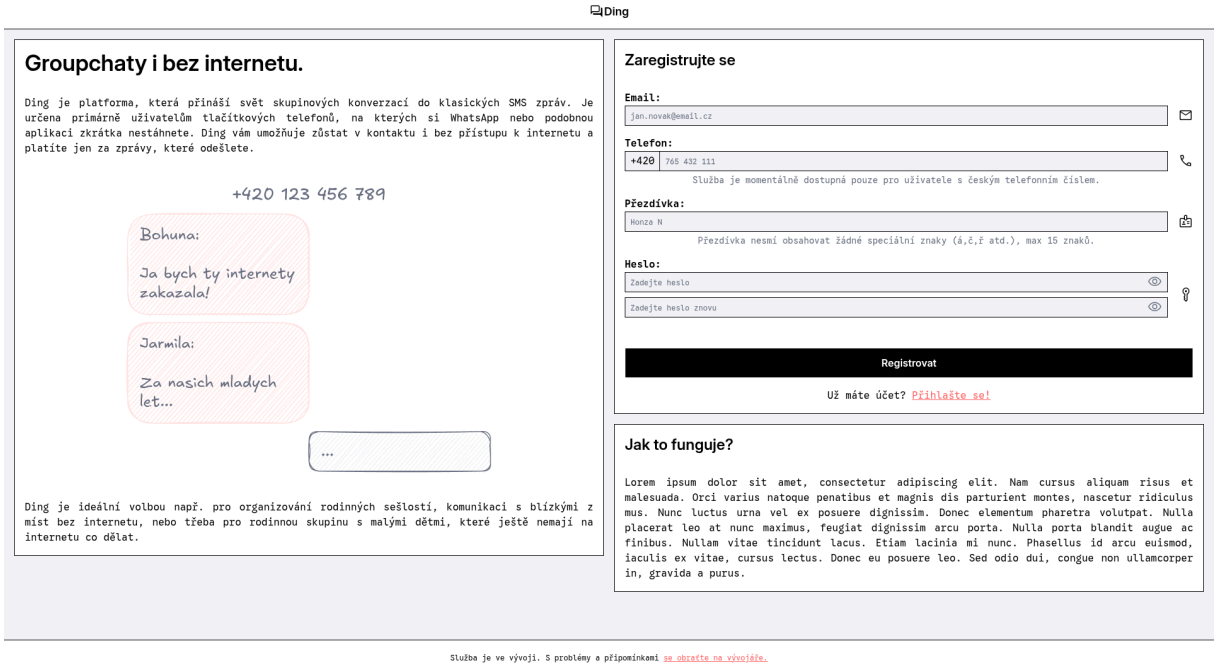
Výrazně mi usnadnily práci také následující knihovny:

- *Tailwind CSS* [6] - framework pro jednodušší práci s CSS pomocí předdefinovaných tříd
- *BetterAuth* [7] - knihovna pro autentizaci a autorizaci uživatelů
- *Drizzle* [8] - ORM pro komunikaci s databází (*SQLite* [9])

3. UI a UX

3.1. Z pohledu uživatele

Uživatel je při navštívení webu přivítán úvodní stránkou, kde nalezne stručný popis služby a panel pro registraci nebo přihlášení.



Obrázek 1: Domovská stránka na PC

Po registraci je požadováno ověření emailové adresy. Uživateli přijde email s odkazem. Dokud neověří svoji emailovou adresu, aplikace ho nepustí dál.

Obrázek 2: Ověřovací email

Ve chvíli, kdy uživatel úspěšně ověří svou emailovou adresu, je přesměrován na hlavní administrační stránku. Ještě než mu je dovoleno spravovat skupiny (vytvářet je nebo se do nich přidávat), musí si ověřit svoje telefonní číslo.

Než si nechá zaslat přivítací zprávu, má ještě možnost si zkontrolovat, že své telefonní číslo zadal správně, popř. ho opravit. Tuto zprávu si může nechat bezplatně zaslat pouze jednou (z důvodu nákladů na její odeslání).

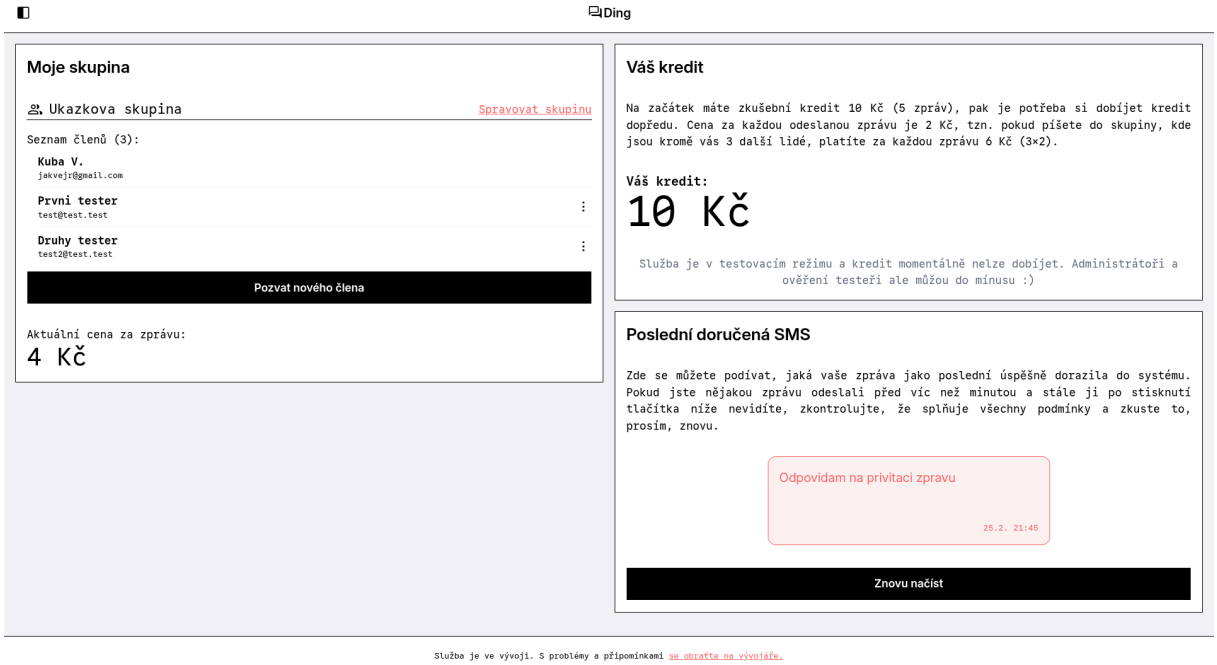
Po potvrzení odeslání přivítací zprávy je uživatel vyzván, aby na ni odpověděl. Zároveň je mu doporučeno si uložit kontakt, ze kterého zprávu obdržel, jelikož skrz tento kontakt bude později komunikovat se svou skupinou.

Obrázek 3: Výzva k odpovědi

Ve chvíli kdy dorazí do systému odpověď od uživatele, je mu zpřístupněna hlavní funkcionalita aplikace. Má možnost buď vytvořit novou skupinu, nebo se přes pozvánku připojit do existující skupiny.

Pozvánky fungují tak, že administrátor skupiny (její zakladatel) zadá email osoby, kterou chce do své skupiny pozvat, a systém automaticky vygeneruje připojovací odkaz a pošle jí ho. Tento odkaz bude fungovat pouze pro účet s daným emailem a je požadováno, aby uživatel měl dokončenou registraci a nebyl v žádné jiné skupině. V případě, že není splněna nějaká z podmínek pro připojení, uživateli je po kliknutí na odkaz vyobrazena stránka popisující potřebné kroky k připojení.

Ve chvíli, kdy je více uživatelů v jedné skupině, služba už je plně funkční. Pokud někdo z nich na systémové číslo pošle zprávu, přijde všem ostatním členům skupiny s připsanou přezdívkou odesílatele.



Obrázek 4: Hlavní stránka z pohledu administrátora skupiny

3.2. Implementace

Veškerá interakce uživatele se systémem probíhá přes webové rozhraní. Jedná se o dynamický web, který komunikuje s databází a v závislosti na jejím obsahu mění UI.

Web je stylizován do jednotného minimalistického stylu. Využívá fonty a ikony z *Google Fonts* [10]. Veškerá stylizace je dělána pomocí Tailwind CSS. Rozhraní je plně responzivní, např. u hlavní stránky se při otevření na mobilním zařízení jednotlivé panely nezarovnávají do dvou sloupců jako na PC, ale pouze do jednoho.

Aplikace je navržena tak, aby uživatele co nejvíc vedla. Vždy mu dá v závislosti na situaci informaci o tom, jaký je potřebný další krok. Zároveň mu také brání v tom, aby některý krok přeskočil, nebo naopak dělal již dokončený krok znovu. Např. před ověřením emailu a telefonního čísla uživateli není zpřístupněna správa skupin a jsou pro něj zablokované veškeré akce s tím související.

4. Backend

4.1. Hosting

Pro fungování služby je nezbytné, aby server běžel pod doménou s HTTPS protokolem. Důvodem je hlavně příjem SMS zpráv, který zajišťuje webhook SMS brány. Tento webhook musí vědět, kam zprávu nasměrovat, dynamická IP adresa tedy není možností, a kdyby komunikace probíhala přes základní HTTP, šla by zpráva internetem nešifrovaná.

Jelikož doma nemám statickou IP adresu, využil jsem pro hosting *Oracle Cloud*. Tam mám virtuální stroj s Linuxem (Ubuntu), který má přiřazenou statickou veřejnou IP adresu. Na tuto adresu pak odkazuji v DNS záznamu své domény, pro kterou mám platný SSL certifikát.

Spouštění a provoz aplikace řeším aplikací *PM2* [11] a provoz webu a SSL certifikát řeším aplikacemi *Nginx* [12] a *Certbot* [13].

4.2. Databáze

Projekt vuužívá SQLite databázi. SQLite umožňuje celou databázi mít pouze v jednom souboru, což snižuje komplexitu ve srovnání s MySQL apod., a pro účely mojí aplikace je toto řešení perfektně dostačující. Komunikaci aplikace s databází zajišťuje ORM *Drizzle*.

Struktura databáze je velmi jednoduchá, jsou v ní tabulky pro:

- Uživatele (*user, accoaunt, session, verification*)
- Skupiny (*group*)
- Pozvánky do skupin (*invite*)

4.2.1. Uživatelé

Tabulky týkající se uživatelů jsou z většiny vygenerované knihovnou *BetterAuth*. Dělal jsem ale několik modifikací v tabulce *user*, viz komentáře ve schématu.

```
export const user = sqliteTable("user", {
  id: text("id").primaryKey(),
  name: text("name").notNull(),
  email: text("email").notNull().unique(),
  emailVerified: integer("email_verified", { mode: "boolean" })
    .default(false)
    .notNull(),
  image: text("image"),
  createdAt: integer("created_at", { mode: "timestamp_ms" })
    .default(sql`(cast(unixepoch('subsecond') * 1000 as integer))`)
    .notNull(),
  updatedAt: integer("updated_at", { mode: "timestamp_ms" })
    .default(sql`(cast(unixepoch('subsecond') * 1000 as integer))`)
    .$onUpdate(() => /* @__PURE__ */ new Date())
    .notNull(),
  phone: text("phone").notNull().unique(),
  groupId: text("group_id").references((): any => group.id), // reference na skupinu,
ve ktere je uzivatel clenem
  welcomeMessageSent: integer("welcome_message_sent", { mode: "boolean" }) // boolean
urcijici, zda uzivateli jiz byla poslana privitaci SMS
    .default(false)
    .notNull(),
  latestMessage: text("latest_message"), // obsah posledni zpravy odeslane uzivatelem
  latestMessageTime: integer("latest_message_time", { mode: "timestamp_ms" }), // cas
posledni zpravy odeslane uzivatelem
  credit: integer("credit").default(10).notNull(), // zustatek uzivatele
});
```

Kód 1: Tabulka *user*

4.2.2. Skupiny

O uchovávání informací o skupinách se stará tabulka *group*, ve které jsou sloupce pro název skupiny a ID jejího administrátora. Dále mám pro tuto tabulku také vytvořenou relaci s tabulkou *user*, pro usnadnění např. dotazů na seznam členů dané skupiny.

```
export const group = sqliteTable("group", {
  id: text("id").primaryKey(),
  name: text("name").notNull(),
  ownerId: text("owner_id").notNull().references((): any => user.id),
});
```

Kód 2: Tabulka *group*

```
export const groupRelations = relations(group, ({ many, one }) => ({
  users: many(user),
  owner: one(user, {
    fields: [group.ownerId],
    references: [user.id],
  }),
}));
```

Kód 3: Relace tabulky *group*

4.2.3. Pozvánky do skupin

Pro každou pozvánku do skupiny se vygeneruje token, který funguje jen pro uživatele s emailem, na který pozvánka byla poslána. Pro ověření, že pozvánka skutečně je určena uživateli, který ji otevřel, je tedy potřeba v tabulce mít také jeho email. Kromě toho je samozřejmě nutné vědět, jaké skupině tato pozvánka přísluší, a zda ještě nebyla použita.

```
export const invite = sqliteTable("invite", {
  token: text("token").primaryKey(),
  email: text("email").notNull(), // email prijemce pozvanky
  groupId: text("group_id").notNull(),
  used: integer("used", { mode: "boolean" })
})
```

Kód 4: Tabulka *invite*

4.3. Komunikace s SMS bránou

4.3.1. Odesílání zpráv

Pro posílání zpráv uživatelům aplikace využívám *HTTP Simple API* od *BulkGate* [14]. Zde je příklad HTTP POST requestu pro odeslání přivítací zprávy:

```
const response = await event.fetch('https://portal.bulkgate.com/api/1.0/simple/
transactional', {
  method: 'POST',
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    "application_id": env.BULKGATE_ID,
    "application_token": env.BULKGATE_TOKEN,
    "number": event.locals.user.phone,
    "text": "Vita Vas Ding :)",
    "country": "cz"
  })
});
```

Kód 5: Odesílání přivítací zprávy

4.3.2. Příjem zpráv

Pro příjem zpráv používám webhook poskytnutý službou *BulkGate*. Endpoint, na který *BulkGate* odkazuji vypadá takto: `https://ding.kubavejrazka.dev/api/sms/receive?secret=<secret>`. Přidaný parametr `secret` zajišťuje, že do systému projdou skutečně jen zprávy, co přijdou z tohoto webhooku.

```
const secret = url.searchParams.get('secret');

if (secret !== env.WEBHOOK_SECRET) {
  return json({ message: 'auth fail' })
}
```

Kód 6: Autorizace příchozích zpráv

4.4. Logika aplikace

Ve chvíli, kdy na endpoint pro příjem zpráv přijde informace, že přišla nová zpráva, systém se podívá, který uživatel ji poslal, v jaké je skupině a jestli splňuje všechny podmínky pro poslání zprávy do skupiny (dostatečný kredit, ověřený účet apod.). Pokud ano, ke zprávě se připíše jeho přezdívka a rozešle se všem ostatním členům skupiny. Uživateli je také struhta z kreditu cena rozeslání.

```
for (const u of targetGroup.users) {
  if (u.id !== senderId) { // brani tomu, aby uzivateli prisla zpet jeho vlastni
zprava
    if (!disableSMS && messageParam!.length <= 140) {
      const response = await fetch('https://portal.bulkgate.com/api/1.0/simple/
transactional', {
        method: 'POST',
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          "application_id": env.BULKGATE_ID,
          "application_token": env.BULKGATE_TOKEN,
          "number": u.phone,
          "text": sender.name + ":\n\n" + messageParam,
          "country": "cz"
        })
      });

      if (!response.ok) {
        const errorDetails = await response.text();
        console.error("Failed to send message to " + u.email + " - API rejection:",
errorDetails);
      } else {
        senderCredit -= 2;
      }
//...
```

Kód 7: Rozeslání zprávy do skupiny

5. Prostor pro zlepšení

Momentálně aplikace má jeden zásadní problém – chaty nejspíše vyrdží jen po omezenou dobu a pak se přesunou na jiné číslo. To je způsobeno tím, že u BulkGate nemám zarezervované vyhrazené telefonní číslo. Mám ho u služby *SmsManager*, se kterou jsem aplikaci začal vytvářet, jenže se ukázalo, že tato služba je téměř nepoužitelná. Měli neustále nějaké technické problémy (zprávy neodcházely nebo nepřicházely, nefungoval mi API klíč atd.) a komunikace s jejich zákaznickou podporou byla katastrofální. Po více než týdnu každodenního kontaktování podpory jsem to tedy vzdal a přesunul se k BulkGate, který sice funguje perfektně, ale nenabízí vyhrazení telefonního čísla. Platforma teď tedy operuje na systémových číslech, kde ale je omezená délka života konverzací.

Mám tedy v plánu se buď vrátit k *SmsManager*, pokud si tu svoji platformu opraví, a nebo se přesunou na nějakou jinou SMS bránu. Našel jsem hned několik alternativ, které by naplnily moje potřeby, ale jsou ve srovnání s *SmsManager* velmi drahé.

Dále bych do aplikace chtěl přidat funkci přiřazení více telefonních čísel jednomu uživateli. To zejména proto, aby uživatelé mohli zaregistrovat např. své prarodiče, kteří jsou primární cílovou skupinou služby, kteří nemusí vůbec mít emailovou adresu a často si nevědí rady s počítači. Myšlenka je tedy taková, že bych si třeba já jako uživatel založil účet (se svým emailem a telefonním číslem) a mohl bych pak přidat profil třeba své babičce, u které by stačilo zadat její telefonní číslo a přezdívku. Její odeslané zprávy bych pak hradil ze svého kreditu.

Aplikace momentálně uživatelům umožňuje být pouze v jedné skupině, při vyhrazení více telefonních čísel by bylo možné ji vylepšit tak, aby jeden uživatel mohl být v několika skupinách zároveň.

Zdroje

[1] "Svelte • Cybernetically enhanced web apps." [Online]. Available: https://svelte.dev/

[2] T. s.r.o, "BulkGate." Accessed: Feb. 28, 2026. [Online]. Available: https://www.bulkgate.com/cs/

[3] "SmsManager.cz." Accessed: Feb. 28, 2026. [Online]. Available: https://smsmanager.cz/

[4] Oracle, "Migration to the Cloud Made Simple." [Online]. Available: https://www.oracle.com/cloud/

[5] Google, "Cloud Computing Services | Google Cloud." [Online]. Available: https://cloud.google.com/

[6] tailwindcss, "Tailwind CSS - Rapidly build modern websites without ever leaving your HTML.." [Online]. Available: https://tailwindcss.com/

[7] "Better Auth." [Online]. Available: https://www.better-auth.com/

[8] "DrizzleORM - next gen TypeScript ORM." [Online]. Available: https://orm.drizzle.team/

[9] "SQLite Home Page." [Online]. Available: https://sqlite.org/

[10] Google, "Google Fonts." [Online]. Available: https://fonts.google.com/

[11] "PM2 - Home." [Online]. Available: https://pm2.keymetrics.io/

[12] "nginx." Accessed: Feb. 28, 2026. [Online]. Available: http://nginx.org/

[13] "Certbot." [Online]. Available: https://certbot.eff.org/

[14] "Specification · BulkGate Helpdesk." Accessed: Feb. 28, 2026. [Online]. Available: https://help.bulkgate.com/docs/en/http-simple-transactional.html